Python

Day-wise Notes

Contents

Day 21 · function as arguments 2

default arguments : . . . 2
keyword arguments : . . . 2
required arguments : . . . 2
return statement : . . . 2
Variable length arguments : . . . 3

Day 22 · data type 4

list . . . 4
check something in python . . . 4
Jump Index . . . 4
list comprehension . . . 5

Day 23 · manipulating list: 6

list.copy() . . . 6
concatenate tuple . . . 6

Day 24 · tuples in python 7

Day 25 · operations i tuple 8

Day 28 · f-strings 9

in old days . . . 9
now the concept of f-string: . . . 9
concept in dictionary further . . . 9

Day 29 · doc string 10

PEP8 . . . 10

Day 30 · recursion 11

find sum of n numbers . . . 11
print n numbers . . . 11
sum of digits:(1234-->1+2+3+4) . . . 12
fibbonacci series:(0,1,1,2,3,5,8.....) . . . 12

Day 31 · set 13

DAY 21

function as arguments

default arguments :

keyword arguments :

required arguments :

Variable length arguments :

default arguments :

default arguments: if any of the argument is not assigned new value it already has a default value

CODE
```
def Code(a=9, c=14):
    print(a+c)

Code(34 , 45)#a=9,b=14 ignored but

Code(34)#a=34, b=14

Code(c=98)#a=9, b=98
```

keyword arguments :

keywords Arguments : order doesnot matter if the given argument is in the way given below

CODE
```
Code(c=46, a=76)#keyword argument
```

required arguments :

CODE
```
def Code2(a, b, c=45):
    print(a+b+c)

Code2(34,45) # here c is default argument but not a & b this are required arguments
```

if not given required arg then error will be there shown

return statement :

return statement : value that is returned and stored whenever the function is called

Variable length arguments :

we can use function argument as list tuple or dictionary

CODE

```
def sum(*numbers):# l=tuple
    pass


def sum(**numbers):# l=dictionary
    pass
```

DAY 22

data type

list

CODE
```
list = [1, 2, 3, 4, 5, 6, 7, 8, 9, "Harry", True]
print(list)
```

list : changeable, ordered , store multiple item in single variable

index : accessing different items from list /

negative indexing ---> positive indexing len(name)-x

CODE
```
 # data type

list = [1, 2, 3, 4, 5, 6, 7, 8, 9, "Harry", True]
print(list)
print(list[1:-4])
```

check something in python

CODE
```
if 4 in list:
    print("Yes")
else:
    print("No")
```

Jump Index

print(list[start:end:jump])

CODE
```
print(list[1:: 3])# [2,5,8,true]jump is skip 1 skip 2 now give three

print(list[1:])#till end
print(list[1:4])#till 4 not including 4th index
print(list[1:-1])#negative indexing
```

list comprehension

on the fly generating a list

there was no list before but as i got need i just started making list

CODE

```
lst = [i for i in range(9)]
#     |-||----------------|
#     |var|       loop     |

print(lst)
```

you can makechanges in variable i.e i*2, i+78

CODE

```
list_2 =[i*2 for i in range(9)]

print(list_2)
```

DAY 23 manipulating list:

CODE
```
list = [1,2,3,4,5,6,7]

print(list)

list.append(7)  # add new elements

list.sort()  # sort the elements ascending order

list.sort(reverse=True)  # ----
#                          #   |---> sort list elements in descending order
list.reverse()  # - ------------

list.index(5)  # index starts fom zero and moves to the 5 and give its output

list.count("#value")  # count the times value inserted comes in it

list.pop(7) #removes 7 from list

list.insert("INDEX", "VALUE")  # inserts the value on the index given below

list.extend("VALUE1", "VALUE2")  # adds multiple value to the list

print(list)
```

list.copy()

CODE
```
list.copy()#this has got a catch in the list

m= list
m[0]=69 #value of the list specific index is changed

print(list)#expected output [1,2,3,4,5] but it comes out to be [69,2,3,4,5]
```

i.e. we use list.copy()

this creates a new list withthe same number of elements in list and now the new changes are there till the new list only

concatenate tuple

concatenate tuple: i.e. simply add two tuple in the output you will get the total no. of elements of both in new elements

DIAGRAM
```
 k(tuple1)--------
                 |
                 |-->l=k+g------> l=new tuple (l has all the elements there in k and g )
                 |
 g(tuple2)--------
```

DAY 24

tuples in python

this is same as list but here the only change is it cannot be changed they are immutable

CODE
```
tuple = (1, 2, 3, 4, "apple", True)
```

we can here print len(tuple), negative indexing , string slicing , check **if 4 in tuple:** , indexing

here the thing that works is same as that of list i.e. tuple jump index or slicing but a catch that this may only work by creating a new tuple

CODE
```
tuple[1:4]

res = tuple.index(3) #gives you first occurence of 3 in tuple

# but if we need to check in a given interval then

res= tuple.count(3, 0,2) #count number of time 3 comes in interval

res=tuple.index(3,2,4) # (value, start , index )

res = len(tuple)#tells you length of tuple

tuple1 = tuple[1:4]
print(tuple)
```

there are more thing to add

DAY 25

operations i tuple

CODE

```
tuple= (1,2,3,4,5)
```

want to add items in tuple

Method 1 :

tuple---> list--->list(add items)---> tuple

Method 2 :

i.e. create a new tuple with items to add add the old and new tuple to add the elements

to get a new operation as output need to create a tuple

CODE

```
res=tuple.count(3) #count number of time 3 comes
```

DAY 28

f-strings

f-strings: method to add variable in print stattements

in old days

CODE

```
a = "Hello My Name {}, I'm from {}"

name = "Dhananjay"
country = "India"

print(a.format(name, country))  # or use index in the place of writing variable names

b = "Hello My Name {1}, I'm from {0}"

print(b.format(country, name))
```

now the concept of f-string:

CODE

```
print(f"my name is {name}, I'm from {country}")

print(f"my name is {{name}}, I'm from {{country}}")
# output : my name is {name, I'm from{country}}
```

concept in dictionary further

in f strings you can use this as :

CODE

```
for key , value in range():
    print(f"Roll Number :{key}, \n marks: {value}")

    # using two variable in string
```

DAY 29

doc string

doc string: it shows the hidden line just beneath the given function

CODE

```
def sum(a,b):
    '''Gives the Output as sum of a and b '''
    print(a+b)

sum(4,5)

print(sum.__doc__)#this gives the line written in comment just below the function defined if any other line than comment comes that is not a doc string
```

PEP8

PEP8: python enhancement proposal ---> way to make a python program more efficient ,readable

simply it is using import in your code

CODE

```
import this
```

DAY 30 recursion

recursion: calling function in same function with different arguments\

Steps to write an recursion ====>

1. first define an function winth name and argument

2. set the base case whenever required

3. then on else acording to the logic write the function which gets you the value

find sum of n numbers

CODE
```
def sum(n):
    if n==1: #base case return this when value is about to end
        return 1
    else:
        return sum(n-1)+n

# sum(4)
print(sum(4))
```

same goes for factorial

CODE
```
def factorial(m):
    if m==0 or m==1:
        return 1
    else :
        return factorial(m-1)*m

print(factorial(5))
```

print n numbers

CODE
```
def print_numbers(n):
    if n==0:
        return # base cap does not further recurse the function
    else:
        print(n)
        return print_numbers(n-1)

print(print_numbers(4))

print("end\n")
```

sum of digits:(1234-->1+2+3+4)

number--> string --> slicing --> integer--> Add

CODE

```
# # sum=[]

# def sum_of_digit_2(n):
#     sum=[]
#     string=str(n)
#     for i in string:
#         # print(i)
#         integer =int(i)
#         # print(type(integer))
#         sum.append(integer)
#     return sum(sum)
# #
# # print(sum)
# # print(sum_of_digit_2(8456))
# sum_of_digit_2(8456)
```

fibbonacci series:(0,1,1,2,3,5,8.....)

CODE

```
def fib_series(n):
    # print(1)
    if n==0 or n==1:
        return 1
    else:
        # print(fib_series(n-2)+fib_series(n-1))
        return fib_series(n-2)+fib_series(n-1)
n=19
for i in range(1,n+1):
    print(fib_series(i) , end =" ")
```

DAY
31

set

set : a collcetion of data in which no item repeat

unodered collection of data

items in set are irreplacable

CODE

```
set1 = {"hello", 45, True}

# empty set: p= set{}

set2 = {}  # dictionary

hello = set()

print(type(hello))
```